

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

SOA4ID: Arquitectura Orientada a Servicios aplicada a sistemas emergentes

Proyecto 3: Justificación de diseño del sistema bancario seguro con DDD

Estudiantes:

Ana Melissa Vásquez Rojas

Estefanny Fabiola Villalta Segura

Manuel Emilio Alfaro Mayorga

Gabriel José Guzmán Rojas

Profesora:

Alejandra Bolaños Murillo

Fecha: 18/06/2026

Contexto del problema

SecureBankito es una entidad financiera que ha identificado dos incidentes de seguridad críticos en su infraestructura tecnológica actual. El primero ocurrió cuando procesos de baja integridad lograron modificar saldos pertenecientes a cuentas de alto valor. Esto fue posible porque un proceso que no contaba con la autorización requerida pudo escribir sobre recursos que excedían su nivel de confianza, constituyendo una violación de los principios establecidos por el modelo de integridad de Biba.

El segundo incidente se relaciona con la confidencialidad de la información. Algunos usuarios con permisos básicos tuvieron acceso a activos y datos clasificados como confidenciales, aun cuando no disponían del nivel de autorización necesario para consultarlos. Esta situación representa una violación del modelo de seguridad Bell-LaPadula, diseñado para proteger la información sensible mediante controles de acceso basados en niveles de clasificación.

Tras el análisis de ambos eventos, se determinó que la causa raíz era que los mecanismos de seguridad se encontraban implementados como una capa externa de la aplicación. Como consecuencia, dichas validaciones podían ser omitidas si un actor accedía directamente a capas inferiores de la arquitectura, comprometiendo tanto la integridad como la confidencialidad de la información.

Para mitigar estos riesgos, se propone migrar hacia una arquitectura basada en microservicios y principios de Domain-Driven Design (DDD). En este enfoque, la seguridad deja de ser un componente externo y pasa a formar parte de las reglas fundamentales del dominio. De esta manera, las restricciones derivadas de los modelos Biba y Bell-LaPadula se integran directamente en la lógica de negocio, garantizando que ninguna operación pueda ejecutarse sin cumplir previamente con las validaciones correspondientes, independientemente del origen de la solicitud o de la capa desde la cual se intente acceder al sistema.

Arquitectura General

A nivel de arquitectura de sistema, SecureBankito está compuesto por microservicios independientes. Cada servicio expone sus propios endpoints HTTP y cuenta con una base de datos PostgreSQL exclusiva, garantizando autonomía, aislamiento de datos y bajo acoplamiento entre dominios.

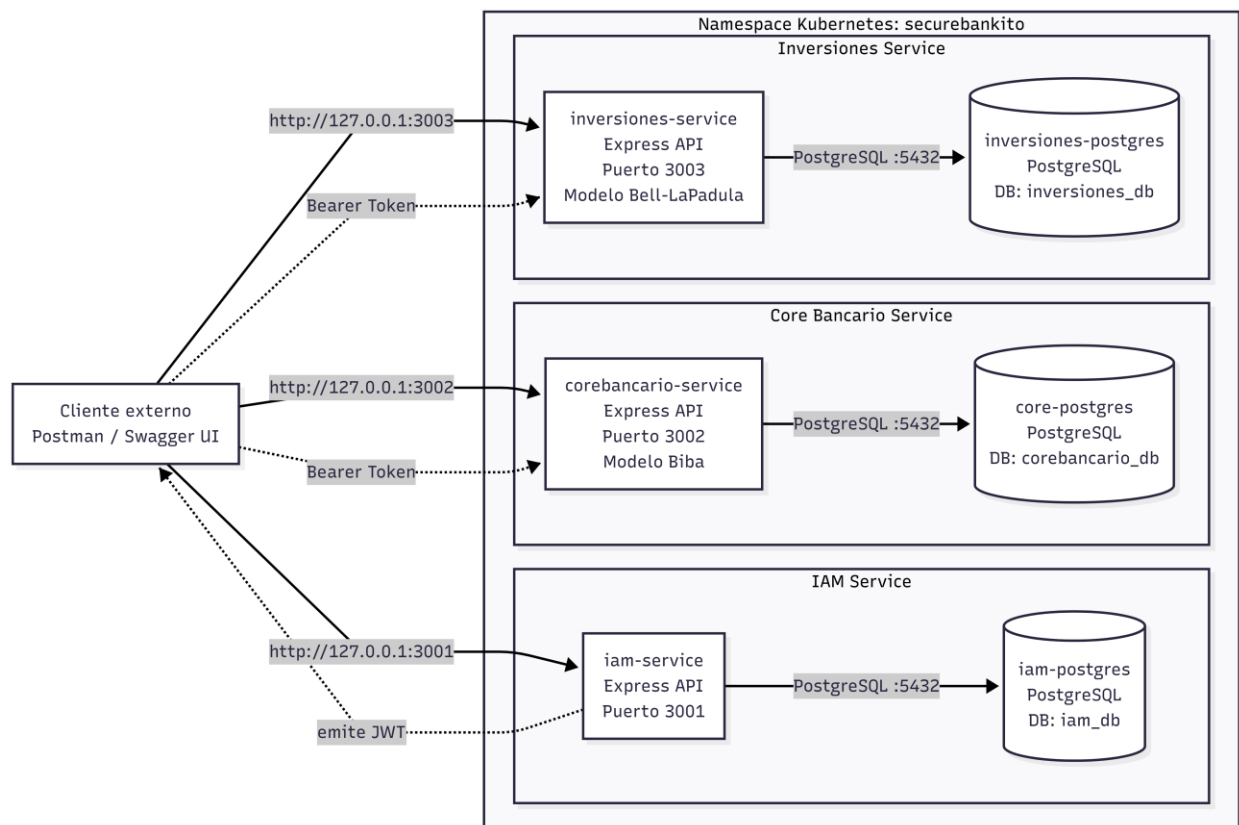


Figura 1. Arquitectura de sistema

A continuación, se presentan los diagramas realizados durante la etapa de diseño de SecureBankito, los cuales muestran los modelos DDD de cada dominio y los flujos de seguridad implementados.

Dominios y Domain-Driven Design.

Puede acceder a los diagramas en el siguiente enlace: [Diagramas DDD](#)

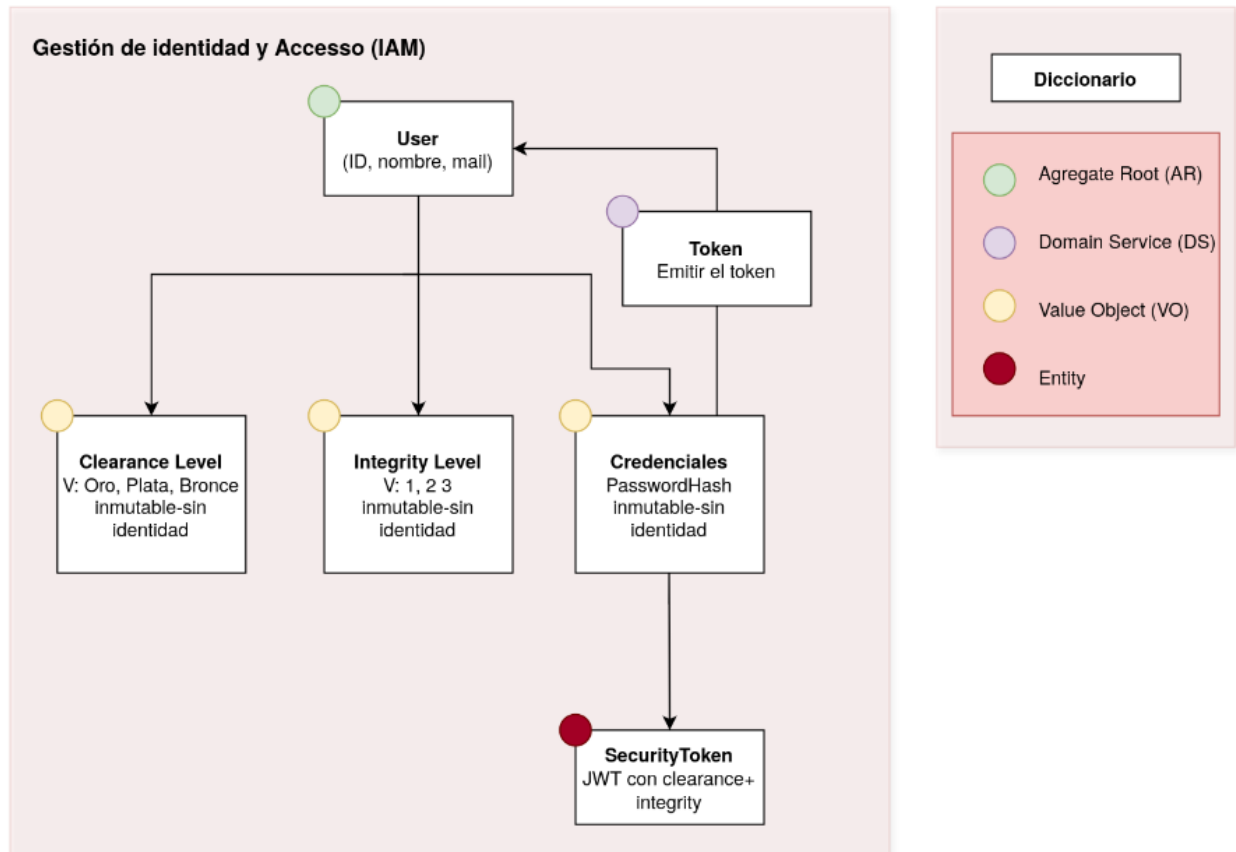


Figura 2. IAM service DDD

El Aggregate Root User es el punto de entrada del bounded context: contiene la identidad (ID, nombre, mail) y agrupa tres Value Objects que describen sus atributos de seguridad. ClearanceLevel (Oro, Plata, Bronce) e IntegrityLevel (1, 2, 3) son inmutables y sin identidad propia.

El Domain Service Token toma al User junto con sus Value Objects y emite el SecurityToken, modelado como Entity porque cada token generado es único en el tiempo (tiene su propio JWT, su propia emisión, su propio ciclo de vida), a diferencia de los VO que solo representan un valor fijo.

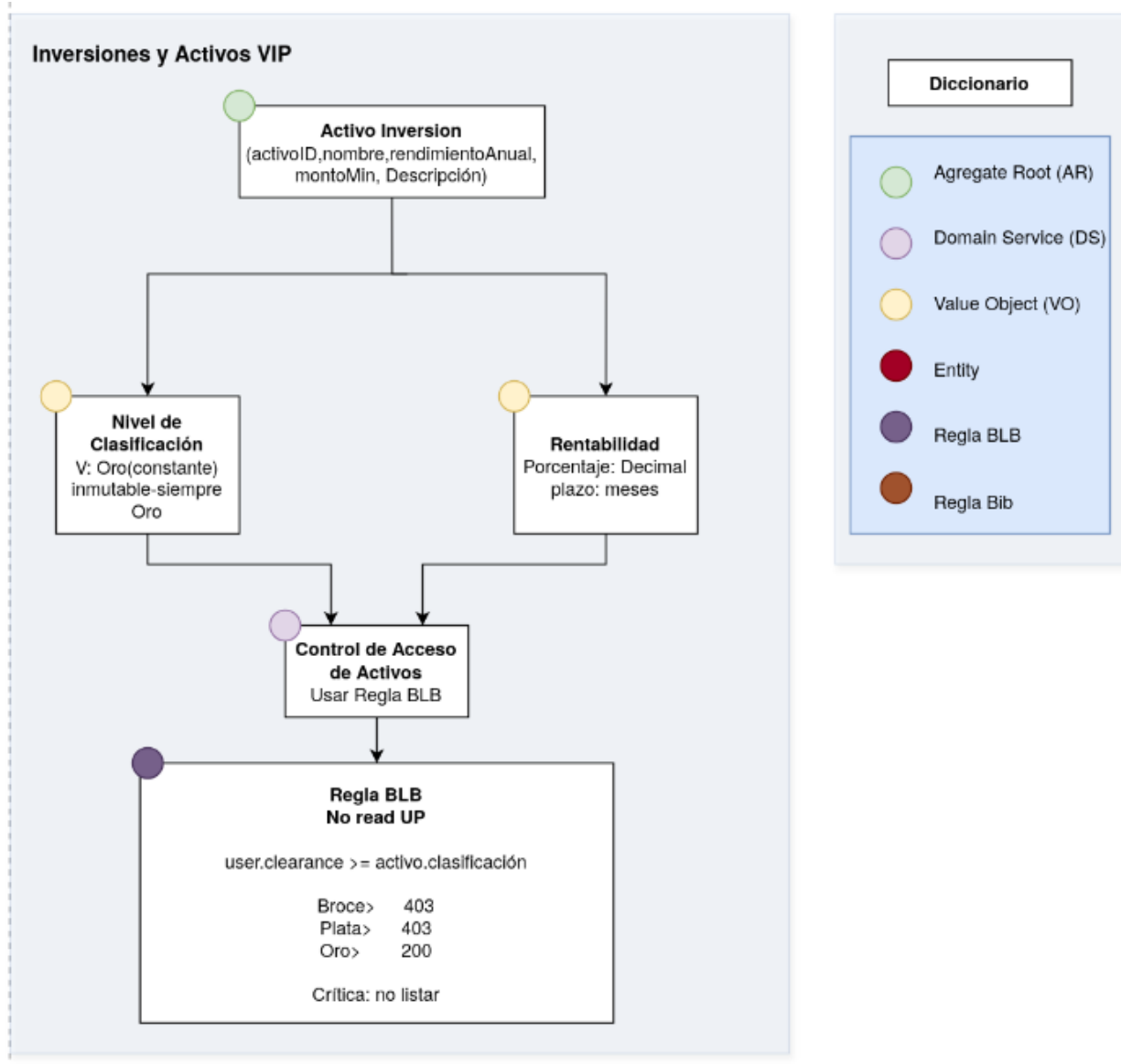


Figura 3. Inversiones service DDD

El Aggregate Root ActivoInversion agrupa la identidad del activo (activoID, nombre, descripción) junto con dos Value Objects. NivelClasificacion es inmutable y siempre vale "Oro"; no es un dato variable sino una constante de diseño que clasifica todo este dominio como de máxima confidencialidad. Rentabilidad (porcentaje + plazo) también es value object porque se compara por valor y no tiene identidad propia.

El Domain Service ControlAccesoActivos consume ambos value objects y aplica la Regla BLP "No Read Up": compara el clearance del usuario contra la clasificacion del activo. Si el clearance es menor, deniega con 403, así es como Bronce y Plata quedan bloqueados y solo Oro obtiene 200. La nota "Crítica: no listar" es clave porque BLP no solo bloquea la lectura del detalle, sino que el activo ni debería aparecer en una lista para alguien sin el clearance suficiente.

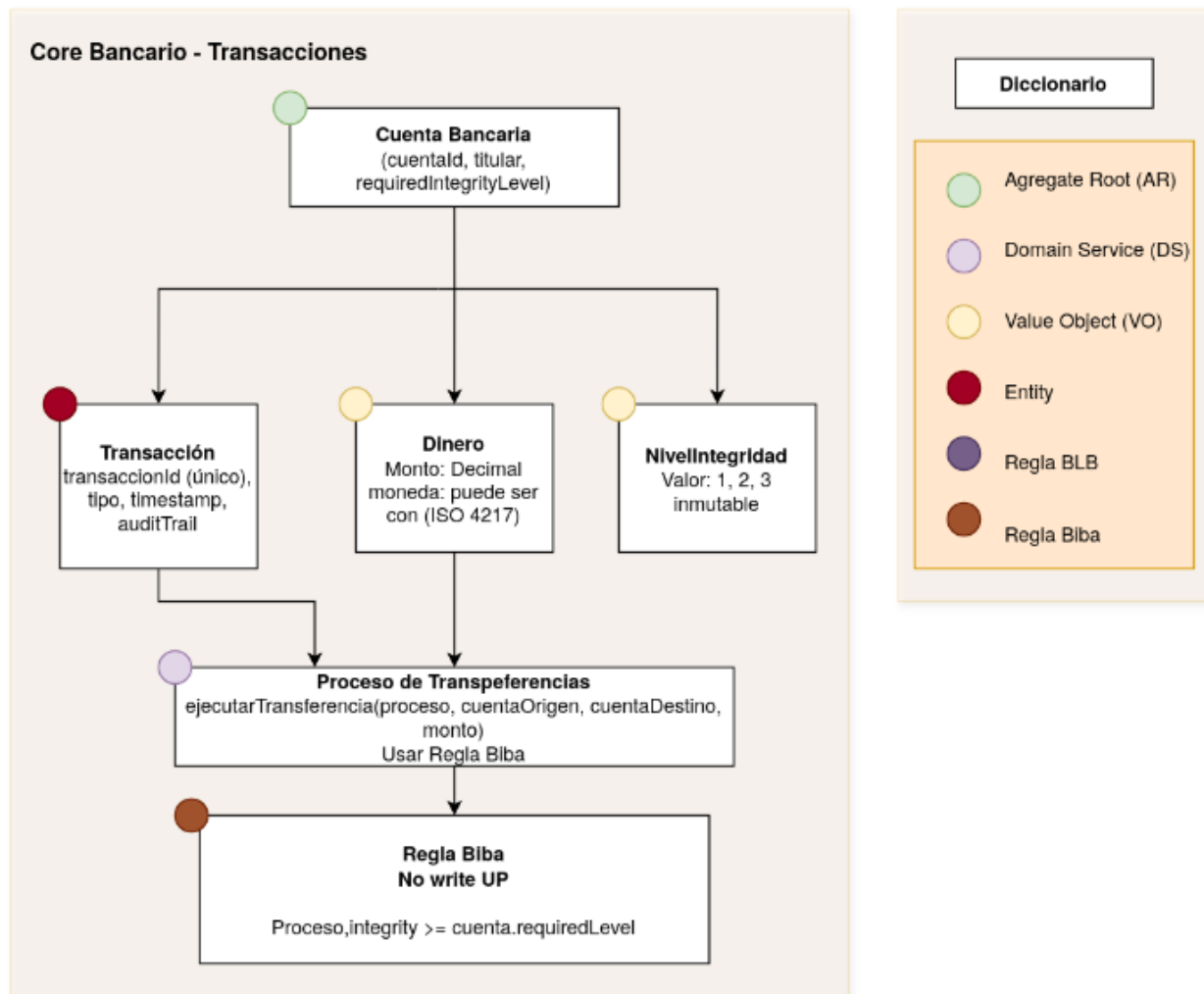


Figura 4. Core Bancario service DDD

El Aggregate Root CuentaBancaria controla el acceso a todo lo que ocurre dentro de la cuenta, incluyendo su requiredIntegrityLevel. De ahí cuelgan tres elementos: Transaccion es Entity, no Value Object, porque tiene un transaccionId único e irrepetible. Dinero y NivelIntegridad son Value Objects: inmutables, comparables por valor, sin ciclo de vida propio.

El Domain Service ProcesoDeTransferencias orquesta la operación ejecutarTransferencia(proceso, cuentaOrigen, cuentaDestino, monto) y aplica la Regla Biba "No Write Up": solo permite escribir si proceso.integrity >= cuenta.requiredLevel. Esto es lo que produce el 403 cuando un proceso Bronce intenta depositar en una cuenta Oro.

Modelos de seguridad Biba y Bell-LaPadula.

Puede acceder a los diagramas de secuencia en el siguiente enlace: [Diagramas de secuencia](#)

Modelo Biba (Integridad)

Core Bancario Service

El modelo Biba protege la integridad de los datos mediante su regla principal de No Write Up: un proceso no puede escribir ni modificar un objeto que tenga un nivel de integridad superior al propio. En el contexto bancario esto es crítico, ya que evita que procesos de baja confianza corrompan saldos de cuentas de alto valor.

En SecureBankito cada cuenta bancaria declara un nivel de integridad requerido para ser modificada, y cada usuario autenticado porta su propio nivel de integridad embebido en el JWT emitido por el IAM. Cuando se ejecuta una transferencia o depósito, el sistema compara ambos valores antes de tocar cualquier saldo.

La implementación sigue una cadena de tres responsabilidades. Primero, el Value Object NivelIntegridad encapsula la comparación y es el único lugar del sistema donde se decide si un proceso tiene suficiente integridad para escribir. Segundo, el Domain Service ProcesadorTransferencias construye el contexto del proceso con la integridad del usuario autenticado y valida tanto la cuenta destino como la cuenta origen antes de ejecutar cualquier movimiento. Si la integridad del proceso es menor a la requerida por la cuenta, lanza una BibaViolationException y los saldos no son modificados en ningún momento. Tercero, el caso de uso EjecutarTransferenciaUseCase captura esa excepción y, antes de retornar el error al cliente, persiste atómicamente una transacción de tipo BLOQUEADA en el audit trail, dejando registro del intento fallido con el nivel de integridad usado y el motivo del bloqueo. Solo entonces retorna HTTP 403.

Un detalle relevante es que el registro de la transacción bloqueada ocurre incluso cuando la operación es denegada, lo que garantiza trazabilidad completa de todos los intentos de violación, no solo de las operaciones exitosas.

Modelo Bell-LaPadula (Confidencialidad)

Inversiones Service

El modelo Bell-LaPadula protege la confidencialidad de los datos mediante su regla principal de No Read Up: un usuario no puede acceder a información clasificada en un nivel superior al de su propio clearance.

En SecureBankito este modelo se aplica en el dominio de Inversiones, donde todos los activos tienen clasificación fija ORO, el nivel más alto de la jerarquía. Esto significa que un usuario con clearance BRONCE o PLATA no solo tiene prohibido ver el contenido de los activos, sino que el sistema ni siquiera confirma su existencia antes de denegar el acceso.

La implementación sigue una cadena de tres responsabilidades encadenadas. Primero, el Value Object NivelClasificacion encapsula la lógica de comparación entre el clearance del usuario y la clasificación del recurso, siendo el único lugar del sistema donde se toma esa decisión. Segundo, el Domain Service ControlAccesoActivos recibe al usuario y al recurso, delega la comparación al Value Object y, si el clearance es insuficiente, lanza una BLPViolationException antes de que cualquier dato sea consultado. Tercero, el caso de uso ListarActivosUseCase garantiza que esta validación ocurra como primer paso absoluto, antes de ejecutar cualquier consulta a la base de datos. Si la excepción es lanzada, se registra la violación en el audit trail y se retorna HTTP 403, sin que el repositorio de activos haya sido tocado en ningún momento.

El resultado práctico es que un usuario ORO accede con normalidad, mientras que un usuario BRONCE o PLATA recibe un 403 sin ninguna información sobre qué existe detrás de ese acceso denegado.

Registros de Decisiones Arquitectónicas (ADRs)

ADR 001: Arquitectura basada en Microservicios con Bounded Contexts DDD

Contexto:

SecureBankito debe garantizar que las reglas de seguridad (Biba y Bell-LaPadula) sean intrínsecas al negocio y no capas externas fácilmente saltables. Además, los tres dominios tienen responsabilidades completamente distintas: gestión de identidad, operaciones financieras y activos de alto valor.

Decisión:

Se separó el sistema en tres microservicios, cada uno representando un Bounded Context distinto bajo los principios de DDD:

- **IAM Service:** Dominio de Identidad y Acceso.
- **Core Bancario Service:** Dominio de Transacciones Financieras.
- **Inversiones Service:** Dominio de Activos VIP.

Cada microservicio tiene su propio modelo de dominio (Aggregates, Entities, Value Objects) y sus propias reglas de negocio de seguridad.

Consecuencias:

Permite que cada dominio evolucione de forma independiente y que las reglas de seguridad sean parte del lenguaje ubicuo de cada contexto. Aumenta la complejidad operativa respecto a un monolito, pero esta complejidad queda encapsulada por el despliegue en Kubernetes. Un proceso que comprometa un dominio no tiene acceso directo a los datos de los otros.

ADR 002: Patrón Base de Datos por Microservicio (Database per Service)

Contexto:

Si los tres dominios compartieran una base de datos, un proceso de baja integridad podría modificar directamente tablas del dominio bancario sin pasar por las validaciones de dominio, violando el modelo Biba en la capa de persistencia.

Decisión:

Se provee una instancia de PostgreSQL completamente separada para cada dominio. Ningún servicio tiene credenciales ni acceso de red directo a la base de datos de otro.

Consecuencias:

Se garantiza el aislamiento de datos entre dominios mediante bases de datos independientes para cada microservicio. Debido a esta separación, no se realizan JOINS entre dominios; cuando se requiere información de más de un dominio, esta debe coordinarse desde la capa de aplicación. En este MVP, la comunicación de seguridad entre servicios se basa en el JWT emitido por IAM, el cual es validado localmente por los demás servicios sin necesidad de llamadas directas entre ellos.

ADR 003: Modelos de Seguridad como Reglas de Dominio (Security by Design)

Contexto:

El enfoque tradicional implementa la seguridad como middleware externo que verifica roles antes de dejar pasar la solicitud. Este enfoque tiene una debilidad fundamental: si un proceso accede directamente a la capa de aplicación o a la base de datos, las reglas de seguridad son ignoradas por completo.

Decisión:

Las reglas de Biba y Bell-LaPadula se implementan dentro de los Domain Services de cada Bounded Context. En el Core Bancario, el ProcesadorTransferencias valida la regla No Write Up antes de ejecutar cualquier movimiento: si la integridad del proceso es menor a la requerida por la cuenta, lanza una BibaViolationException que se traduce en un HTTP 403 sin que ningún saldo sea modificado. En el servicio de Inversiones, el ControlAccesoActivos valida la regla No Read Up antes de retornar cualquier activo: si el clearance del usuario no alcanza el nivel ORO, lanza una BLPViolationException con el mismo resultado. En ambos casos, las reglas se aplican de forma obligatoria, ya que los Domain Services centralizan las operaciones que modifican el estado de los Aggregates.

Consecuencias:

La seguridad formal queda acoplada al dominio de manera intencional. Cualquier ruta de código que intente modificar un saldo o listar un activo necesariamente pasa por el Domain Service correspondiente. El costo es que las reglas de seguridad están distribuidas entre dominios y no centralizadas en un solo componente.

ADR 004: JWT como portador de Etiquetas de Seguridad**Contexto:**

Los servicios de Core Bancario e Inversiones necesitan conocer el nivel de clearance e integridad del proceso que hace una solicitud para poder aplicar las reglas de Biba y Bell-LaPadula. Consultar al IAM Service en cada request introduciría latencia y un punto de fallo sincrónico.

Decisión:

El TokenService del IAM inserta las etiquetas de seguridad directamente en el payload del JWT al momento del login:

Json

```
{  
  
  "sub": "<userId>",  
  
  "username": "usuario_bronce",  
  
  "clearance": "BRONCE",  
  
  "integrity": 1  
  
}
```

El JWT se firma con un secreto compartido (JWT_SECRET). Los demás servicios validan la firma localmente y extraen clearance e integrity sin necesidad de llamar al IAM en cada request.

Consecuencias:

Comunicación entre servicios desacoplada y sin latencia adicional. La desventaja es que si se revoca un usuario, su token sigue siendo válido hasta que expire (15 minutos). Para el alcance de esta PoC, este tradeoff es aceptable.

ADR 005: Ausencia de API Gateway

Contexto:

En arquitecturas de microservicios es común centralizar el enrutamiento y autenticación en un API Gateway. En este proyecto cada cliente (Postman, scripts de prueba) consume los servicios directamente por su puerto.

Decisión:

No se implementa API Gateway. Cada microservicio expone su propio puerto y valida el JWT de forma independiente. La autenticación está centralizada únicamente en el IAM como emisor del token, la autorización ocurre dentro del dominio de cada servicio.

Consecuencias:

Reduce la complejidad operativa para el alcance de esta PoC. El cliente debe conocer los tres puertos de los servicios. Esta decisión es adecuada para demostrar que las reglas de seguridad son intrínsecas al dominio y no dependen de una capa de infraestructura externa para funcionar.

ADR 006: NivelIntegridad y ClearanceLevel como Value Objects (inmutables)

Contexto:

El objetivo del sistema es comparar niveles de seguridad entre procesos y recursos. Es necesario definir si estos niveles son Entities (con identidad propia y ciclo de vida) o Value Objects (definidos únicamente por su valor).

Decisión:

NivelIntegridad, ClearanceLevel, Dinero y Rentabilidad se implementan como **Value Objects** con las siguientes características:

- **Inmutables:** todos utilizan Object.freeze(this) al construirse. No existe ningún método que modifique el estado interno del objeto.
- **Igualdad por valor:** dos instancias de ClearanceLevel('ORO') son equivalentes; no importa cuál de las dos se usa para comparar.
- **Sin identidad propia:** no tienen un ID en base de datos. Existen como parte del Aggregate que los contiene.
- **Encapsulan lógica de comparación:** ClearanceLevel.isAtLeast(other) y NivelIntegridad.esSuficienteParaEscribir(other) evitan que la lógica de comparación de niveles quede dispersa en capas de aplicación o infraestructura.

En cambio, User, CuentaBancaria, ActivoInversion y Transaccion son **Entities** porque:

- Tienen identidad propia ('userId', 'cuentaId', 'activoId', 'transaccionId') que persiste en base de datos.
- Su igualdad se determina por ID, no por el valor de sus atributos.
- Tienen un ciclo de vida (se crean, se modifican, se consultan a lo largo del tiempo).

Consecuencias:

El motor de reglas de Biba y Bell-LaPadula opera directamente sobre Value Objects, lo que hace las comparaciones de niveles de seguridad predecibles, testeables y sin efectos secundarios. La inmutabilidad garantiza que un nivel de seguridad no pueda ser modificado en memoria durante la ejecución de una transacción.

ADR 007: Transaccion como Entity con Audit Trail immutable

Contexto:

Cada movimiento financiero debe quedar registrado con un rastro de auditoría que incluya quién lo ejecutó, con qué nivel de integridad y cuál fue el resultado. Este registro no debe ser modificable una vez creado.

Decisión:

Transacción se implementa como una Entity generada en el momento en que ProcesadorTransferencias ejecuta el movimiento. Una vez persistida, no existe ningún caso de uso que la modifique. El campo auditTrail registra el contexto de la operación (cuenta origen, cuenta destino, tipo de operación) y el campo integrityUsada registra el nivel Biba del proceso que autorizó el movimiento.

Consecuencias:

El historial de transacciones es un registro fiel y no repudiable de cada operación. Permite demostrar, en la PoC, que una transferencia bloqueada por Biba nunca genera un registro de EXITOSA en la base de datos del dominio bancario.